UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

I. General Statement of the Problem

a. What is the application about? Who are the target users? What problem does the application solve?

Le Festin is a recipe and meal plan management system designed to help users organize recipes, manage ingredients, and plan meals efficiently. The target users of the application include students, busy professionals, families, home cooks, and anyone interested in tracking their meals, recipes, or pantry ingredients in a convenient and organized way.

The application solves common problems such as difficulty in organizing recipes, forgetting available pantry ingredients, wasting food due to unused ingredients, and struggling to plan meals ahead of time. By combining recipe management, ingredient tracking, and meal planning features into one system, Le Festin helps users save time, reduce food waste, and make meal preparation more efficient and convenient.

II. Database Requirements

A. E/R Diagram: Design of the database schema.

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

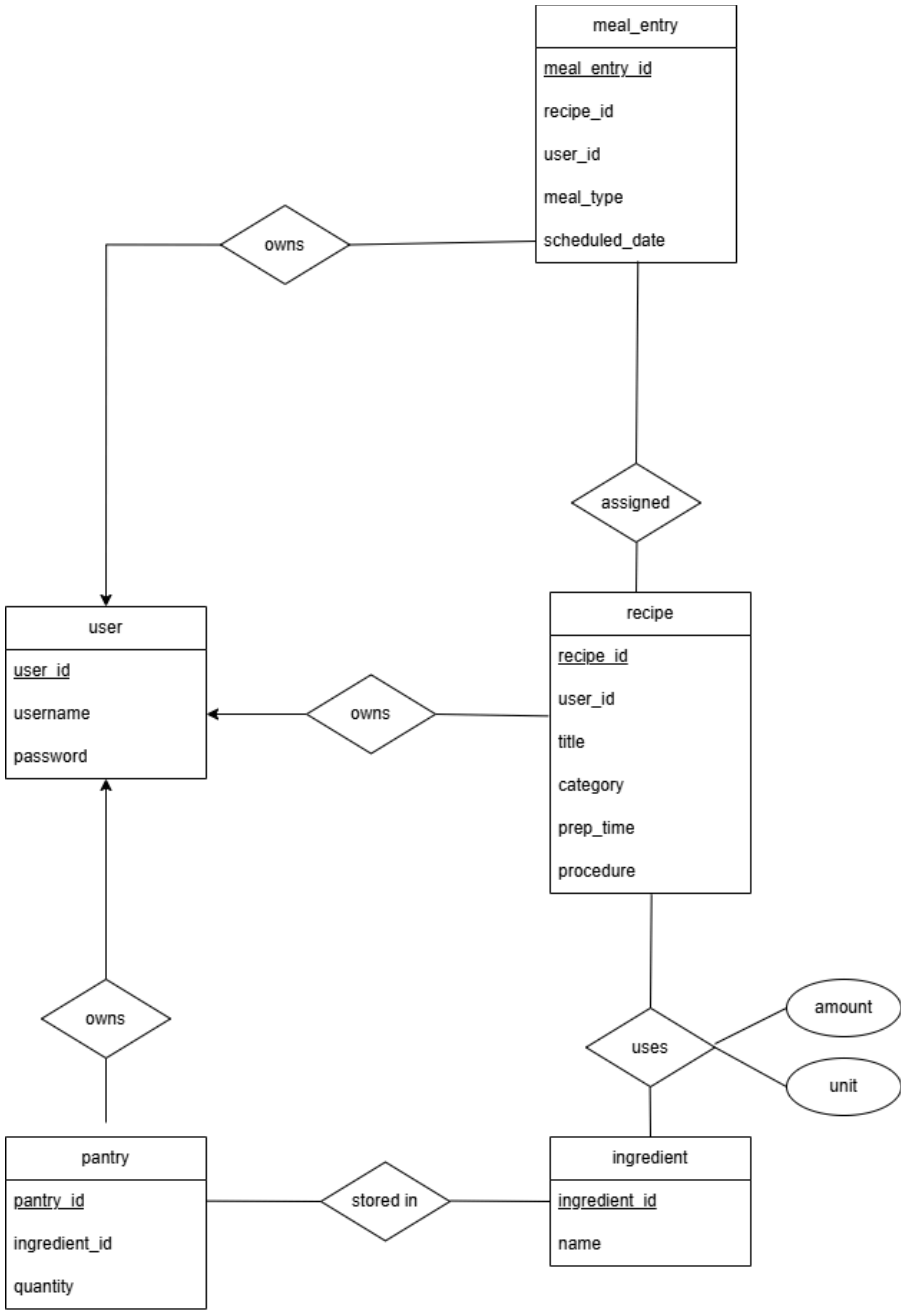

Figure 1. E/R Diagram of Le Festin

B. Technical Description: Explanation of the database requirements and how they relate to the E/R diagram.

1. Structural Hierarchy and User Ownership

The system employs a centralized ownership architecture where the User entity serves as the primary anchor for all data.

Requirement Implementation: The narrative specifies that each user maintains isolated collections. The User entity (containing user_id, username, and password) is connected via "owns" relationship diamonds to the Recipe, Pantry, and Meal_Entry entities. This ensures that while a single user can possess multiple records across the system, each record is strictly mapped to one unique identifier, preventing cross-user data leakage.

2. Recipe Specification and Content

The Recipe entity functions as a static template for culinary instructions.

Requirement Implementation: The system requires unique identification and descriptive metadata including title, category, and preparation instructions. These are modeled as attributes of the Recipe entity: recipe_id, title, category, prep_time, and procedure.

3. Ingredient Composition

The relationship between Recipe and Ingredient represents the most complex logical intersection in the schema.

Requirement Implementation: Ingredients must be reusable across multiple recipes, and recipes must support multiple ingredients. Additionally, the system must record unique measurements (quantity and unit) for each instance. The amount and unit are placed as attributes on the relationship itself rather than the entities. This signifies that these values only exist when a specific ingredient is assigned to a specific recipe.

4. Pantry Inventory and Stock Management

The Pantry entity acts as a junction to track physical inventory without duplicating master ingredient data.

Requirement Implementation: The pantry must link a specific user to a specific ingredient while tracking current volume. The Pantry entity contains the quantity attribute and is linked to the User via an "owns" relationship and to the Ingredient via the "stored in" relationship. This design allows the system to differentiate between a Master Ingredient (the concept of salt) and a Pantry Item (the 500g of salt in a user's cabinet).

5. Dynamic Meal Planning and Overrides

The Meal_Entry entity provides a temporal layer to the database, allowing for scheduling and categorization.

Requirement Implementation: Users must organize recipes into daily, weekly, or monthly plans and have the ability to redefine the meal type for that specific instance. The Meal_Entry entity contains meal_type and scheduled_date.

C. **Relation Schema**: Conversion of the E/R diagram into relational schemas, including primary and foreign keys.

```
SQL
user(user_id, username, password_hash) PK: user_id

ingredient(ingredient_id, name) PK: ingredient_id

recipe(recipe_id, user_id, title, category, prep_time, procedure)
PK: recipe_id
FK: user_id → user(user_id)

recipe_ingredient(recipe_id, ingredient_id, quantity, unit)
PK: recipe_id, ingredient_id
FK: recipe_id → recipe(recipe_id), ingredient_id → ingredient(ingredient_id)

pantry(ingredient_id, user_id, quantity, unit)
PK: ingredient_id, user_id
FK: ingredient_id → ingredient(ingredient_id), user_id → user(user_id)

meal_entry(user_id, meal_type, scheduled_date, recipe_id)
PK: user_id, meal_type, scheduled_date
FK: recipe_id → recipe(recipe_id), user_id → user(user_id)
```

III. Implementation

A. **SQL Table Definition**

```
SQL
-- ── Database ──────────────────────────────────────────────
CREATE DATABASE IF NOT EXISTS la_festin
    CHARACTER SET utf8mb4
```

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

```
    COLLATE utf8mb4_unicode_ci;

USE la_festin;

-- ── Safety: drop in reverse dependency order ──────────────
-- Child tables first, parent tables last.
-- Prevents FK constraint errors on re-run.
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS meal_entry;
DROP TABLE IF EXISTS pantry;
DROP TABLE IF EXISTS recipe_ingredient;
DROP TABLE IF EXISTS recipe;
DROP TABLE IF EXISTS ingredient;
DROP TABLE IF EXISTS user;
SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================
--  TABLE 1: user
--  No dependencies — created first.
-- ============================================================
CREATE TABLE user (
    user_id       INT          NOT NULL AUTO_INCREMENT,
    username      VARCHAR(50)  NOT NULL,
    password_hash VARCHAR(255) NOT NULL,

    CONSTRAINT pk_user     PRIMARY KEY (user_id),
    CONSTRAINT uq_username UNIQUE      (username)
);

-- ============================================================
--  TABLE 2: ingredient
--  No dependencies — created alongside user.
-- ============================================================
CREATE TABLE ingredient (
    ingredient_id INT          NOT NULL AUTO_INCREMENT,
    name          VARCHAR(100) NOT NULL,

    CONSTRAINT pk_ingredient PRIMARY KEY (ingredient_id),
    CONSTRAINT uq_ingredient UNIQUE      (name)
);
```

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

```sql
-- ============================================================
--  TABLE 3: recipe
--  Depends on: user
-- ============================================================
CREATE TABLE recipe (
    recipe_id  INT         NOT NULL AUTO_INCREMENT,
    user_id    INT         NOT NULL,
    title      VARCHAR(150) NOT NULL,
    category   ENUM(
                 'Breakfast',
                 'Lunch',
                 'Dinner'
               )           NOT NULL,
    prep_time  INT         NOT NULL,
    `procedure`  TEXT      NOT NULL,

    CONSTRAINT pk_recipe       PRIMARY KEY (recipe_id),
    CONSTRAINT fk_recipe_user  FOREIGN KEY (user_id)
        REFERENCES user (user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_prep_time   CHECK (prep_time > 0)
);

-- ============================================================
--  TABLE 4: recipe_ingredient
--  Depends on: recipe, ingredient
-- ============================================================
CREATE TABLE recipe_ingredient (
    recipe_id     INT           NOT NULL,
    ingredient_id INT           NOT NULL,
    quantity      DECIMAL(10, 2) NOT NULL,
    unit          VARCHAR(50)   NOT NULL,

    CONSTRAINT pk_recipe_ingredient
        PRIMARY KEY (recipe_id, ingredient_id),
    CONSTRAINT fk_ri_recipe
        FOREIGN KEY (recipe_id)
        REFERENCES recipe (recipe_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
```

```
    CONSTRAINT fk_ri_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient (ingredient_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_ri_quantity CHECK (quantity > 0)
);

-- ============================================================
--  TABLE 5: pantry
--  Depends on: ingredient, user
-- ============================================================
CREATE TABLE pantry (
    ingredient_id INT            NOT NULL,
    user_id       INT            NOT NULL,
    quantity      DECIMAL(10, 2) NOT NULL,
    unit          VARCHAR(50)    NOT NULL,

    CONSTRAINT pk_pantry PRIMARY KEY (ingredient_id, user_id),
    CONSTRAINT fk_pantry_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient (ingredient_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_pantry_user
        FOREIGN KEY (user_id)
        REFERENCES user (user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_pantry_qty CHECK (quantity >= 0)
);

-- ============================================================
--  TABLE 6: meal_entry
--  Depends on: recipe, user
-- ============================================================
CREATE TABLE meal_entry (
    recipe_id      INT  NOT NULL,
    user_id        INT  NOT NULL,
    meal_type      ENUM(
                     'Breakfast',
```

```
                    'Lunch',
                    'Dinner'
                )     NOT NULL,
    scheduled_date DATE NOT NULL,

    CONSTRAINT pk_meal_entry
        PRIMARY KEY (user_id, scheduled_date, meal_type),
    CONSTRAINT fk_me_recipe
        FOREIGN KEY (recipe_id)
        REFERENCES recipe (recipe_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_me_user
        FOREIGN KEY (user_id)
        REFERENCES user (user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- ============================================================
--  INDEXES
-- ============================================================
CREATE INDEX idx_recipe_user  ON recipe          (user_id);
CREATE INDEX idx_ri_recipe    ON recipe_ingredient(recipe_id);
CREATE INDEX idx_pantry_user  ON pantry           (user_id);
CREATE INDEX idx_me_user_date ON meal_entry       (user_id, scheduled_date);
```

B. Sample Queries

JOIN Queries (Multi-Table)

SQL

Recipe Ingredients with Names - RecipeIngredientDAO.java:

```
SELECT ri.recipe_id, ri.ingredient_id, ri.quantity, ri.unit, i.name AS
ingredient_name
FROM recipe_ingredient ri
JOIN ingredient i ON ri.ingredient_id = i.ingredient_id
WHERE ri.recipe_id = ?
ORDER BY i.name ASC
```

```
Purpose: Returns all ingredients for a recipe with ingredient names, avoiding a
second query per ingredient.
```

```
SQL
Pantry with Ingredient Names - PantryDAO.java:
SELECT p.ingredient_id, p.user_id, p.quantity, p.unit, i.name AS ingredient_name
FROM pantry p
JOIN ingredient i ON p.ingredient_id = i.ingredient_id
WHERE p.user_id = ?
ORDER BY i.name ASC

Purpose: Shows user's pantry with human-readable ingredient names.
```

```
SQL
Meal Entries with Recipe Details - MealEntryDAO.java (base fragment):
SELECT me.recipe_id, me.user_id, me.meal_type, me.scheduled_date,
       r.title AS recipe_title, r.category AS recipe_category
FROM meal_entry me
JOIN recipe r ON me.recipe_id = r.recipe_id
WHERE me.user_id = ? AND me.scheduled_date BETWEEN ? AND ?

Purpose: Populates weekly planner without requiring separate recipe lookups.
```

Range Queries (BETWEEN)

```
SQL
Weekly Planner Range - MealEntryDAO.java:
WHERE me.scheduled_date BETWEEN ? AND ?
ORDER BY me.scheduled_date, me.meal_type

Purpose: Efficiently loads entire week (or month) of meals in one query.
```

Advanced Query Patterns

SQL

INSERT ON DUPLICATE KEY UPDATE (Upsert) - PantryDAO.java:

```
INSERT INTO pantry (ingredient_id, user_id, quantity, unit)
VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
    quantity = VALUES(quantity),
    unit = VALUES(unit)
```

Purpose: Atomically adds a pantry item or updates quantity if it already exists (no race condition between check and insert).

SQL

Existence Checks (COUNT) - PantryDAO.java:

```
SELECT COUNT(*) FROM pantry
WHERE ingredient_id = ? AND user_id = ?
```

Purpose: Efficient boolean check before insert to avoid constraint violations.

Transactions

SQL

Meal Entry Swap - MealEntryDAO.java

```
connection.setAutoCommit(false);
try {
    deleteEntry(first);      // DELETE 1
    deleteEntry(second);     // DELETE 2
    addEntry(swapped1);      // INSERT 1
    addEntry(swapped2);      // INSERT 2
    connection.commit();
} catch (SQLException e) {
    connection.rollback();
    throw e;
} finally {
    connection.setAutoCommit(previousAutoCommit);
}
```

Purpose: Ensures all-or-nothing atomicity when swapping two occupied meal slots. If any step fails, all changes roll back.

C. **Frontend (GUI) Development**:

The frontend of Le Festin was developed using Java Swing, but the interface was designed to feel more modern and organized than a typical desktop application. Instead of opening multiple windows, the system uses a CardLayout to switch smoothly between pages like the Recipe List and Pantry while keeping the sidebar and header fixed. This creates a cleaner and more seamless user experience.

IV. Conclusion and Reflection

Building Le Festin from scratch was not an easy feat, from brainstorming to implementation. We had to think through so many layers — the database schema, the data access objects (DAOs), the business logic, and the UI. These pieces had to fit perfectly, and the set up took time.

The biggest challenge we faced was getting on track, a group member was sick, we had insufficient people and time to work. We had to take almost a week-worth of backlogs individually to finish within the remaining weeks. Next, managing the database connections between UI and DAO. Debugging database issues is slow because we have to trace from the UI all the way down to the SQL. The recipe matching algorithm was also tricky. Figuring out how to calculate which recipes matched the ingredients someone had in their pantry involved a lot of thinking. It's not hard math, but getting it to feel right and work fast took patience. Lastly, the UI was surprisingly complex too, having multiple panels talking to each other and state management became messy sometimes. We learned that small organizational choices at the start matter a lot.

We learned that structure matters more than we thought. Separating concerns into layers — models, DAOs, services, and UI made things easier. Testing small pieces as we built them would have saved us hours.

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

V. Appendixes

A. User Manual for each feature

FEATURE	STEPS
Authentication (Login / Register)	1. Launch the app – the Login dialog appears. 2. To sign in: enter Username and Password → click “Sign In”. 3. To create an account: click "Register", fill username/password/confirm, click "Create Account". 4. On success the main window opens and your username appears top-right.
Recipes (Browse / View / Add / Edit / Delete)	1. Open "Recipes" (default view). 2. Search by title or category using the search field. 3. Add a recipe: ➢ Click (+) in header. ➢ Fill Title, Total time, Category, Steps. ➢ Add ingredients: click "+ Add ingredient", set quantity/unit/name (use the unit dropdown). ➢ Click "Save Recipe". 4. Edit: click the pencil icon on a recipe card → update → Save. 5. Delete: click the ✖ icon on a card → confirm deletion. 6. View details: click anywhere on a recipe card to see ingredients and steps; use Back to return.
Pantry (My Pantry)	1. Open "My Pantry" from sidebar. 2. Add ingredient: click "Add" → the Add/Edit Ingredient dialog opens → enter name, qty, unit → Save. 3. Edit: select a row → click "Edit" → modify → Save. 4. Remove: select a row → click "Remove" → confirm. 5. To find recipes you can cook: click "Match Recipes" in the search area to jump to Suggestions.
Recipe Suggestions (Recipe Matching)	1. Open "Suggestions" from sidebar. 2. Click "Refresh" or let the panel calculate matches (it loads on view). 3. Use filter buttons: All / Ready to Cook (100%) / Partial Match. 4. Review each card: progress bar shows percent matched; missing ingredients listed. 5. Assign a recipe to the meal planner: click "Assign to Plan" on a suggestion card.
Weekly Meal Planner	1. Open "Meal Planner". 2. Navigate weeks with Prev / Next. 3. Assign a recipe:

	➢ Click a slot → the assign dialog opens → choose recipe → Save. ➢ Or assign from a suggestion card "Assign to Plan". 4. Drag to move/swaps: click-and-drag a filled slot to another slot to move or swap. 5. Auto-Generate Week: click "Auto-Generate Week" to fill slots automatically (uses internal generator if present). 6. Clear Week: click "Clear Week" to remove all entries for current week (confirm). Export week to CSV: click "Export CSV" (file chooser opens).
Grocery List	1. Open "Grocery List". 2. Use the "From" and "To" date spinners (defaults to current Monday–Sunday). 3. Click "Generate List". 4. Results appear in a table: Ingredient | Quantity | Unit. 5. Export CSV: click "Export CSV" → choose file name/location (ensures .csv extension). 6. Print: click "Print" to send the table to a printer.
CSV Export (Meal Plan & Grocery)	1. From Grocery List: after generating, click "Export CSV". 2. From Weekly Planner: click "Export CSV" in planner nav. 3. In file chooser pick location; the app appends .csv if missing. 4. On success you see a confirmation with file path.

B. **Application setup/installation instructions**

🛠️ Installation & Setup

Follow these steps to get a local development environment running.

1. Prerequisites

Ensure you have the following installed on your system:

- Java Development Kit (JDK) 17+: This project uses Java 17 features.
- Maven: For dependency management and building the project.
- MySQL Server 8.4+: To host the application database.

2. Database Configuration

1. Open your MySQL terminal or GUI (like MySQL Workbench).
2. Create a new database:

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

```SQL
CREATE DATABASE la_festin;
```

Use schema.sql

```SQL
mysql -u your_username -p la_festin < path/to/schema.sql
```

4. *(Optional)* **Seed Data:** To populate the database with initial recipes and ingredients for testing, run the seed utility:

```Shell
mvn compile exec:java
"-Dexec.mainClass=com.lafestin.config.SeedDataTest"
```

3. Application Environment
The app requires a `config.properties` file in the `src/main/resources` directory to connect to your database.
1. Locate `config.properties.example` in the root or resources folder.
2. Duplicate it and rename it to `config.properties`.
3. Update the values with your local credentials:

```properties
db.url=jdbc:mysql://localhost:3306/la_festin
db.user=your_mysql_user
db.password=your_mysql_password
```

4. Building the Application
Use Maven to clean the project and package it into an executable "fat" JAR (which includes all dependencies like the MySQL connector).

```shell
mvn clean package
```

This will generate two files in the `/target` directory. Look for `la-festin-1.0-SNAPSHOT-fat.jar`.

5. Running the App

You can launch the GUI directly from the terminal:

Bash

```
java -jar target/la-festin-1.0-SNAPSHOT-fat.jar
```

Testing

To ensure the DAO and Service layers are functioning correctly with your database:

Bash

```
mvn test
```
